



UNIVERSITÉ DE PERPIGNAN VIA DOMITIA

Master Calcul Haute Performance & Simulation

Master CHPS, 2ème année, semestre 3

HPC pour l'Intelligence Artificielle

Reconnaissance des panneaux de signalisation (GTSRB)

Réalisé par :

Salma BENSMAIL

Master 2 CHPS

Enseignante :

Mme **Amira MOUAKHER**

UE HPC pour l'Intelligence Artificielle

Remerciements

Je tiens à remercier chaleureusement Mme **Amira MOUAKHER** pour son encadrement dans le cadre de l'UE « HPC pour l'Intelligence Artificielle ». les bonnes pratiques de programmation scientifique et la mise en œuvre de réseaux de neurones convolutionnels ont été déterminantes pour la réalisation de ce projet.

Je la remercie également pour la qualité de son enseignement tout au long du semestre. Ce projet a été pour moi l'occasion d'appliquer concrètement les notions de calcul haute performance abordées dans son cours et de renforcer notre intérêt pour la vision par ordinateur.

Table des matières

1	Introduction	3
1.1	Objectifs du projet	3
2	Jeu de données GTSRB	3
2.1	Description générale	3
2.2	Prétraitement des données	4
3	Méthodologie et architecture du modèle	4
3.1	Pipeline général	4
3.2	Architecture du réseau de neurones convolutionnel	4
3.3	Justification des choix techniques	6
3.4	Hyper-paramètres d'entraînement	6
4	Résultats expérimentaux	7
4.1	Évolution de l'accuracy et de la loss	7
4.2	Performances globales sur le test	7
4.3	Matrice de confusion et analyse des confusions	7
4.4	Exemples qualitatifs de prédictions	8
4.5	Démo d'inférence sur une image du jeu de test	10
5	Analyse critique et pistes d'amélioration	10
6	Implémentation et aspects HPC	11
7	Conclusion	11

1 Introduction

Dans le cadre du module « HPC pour l'Intelligence Artificielle », notre objectif est de concevoir un modèle de deep learning basé sur un réseau de neurones convolutionnel (CNN) pour résoudre un problème réel de vision par ordinateur. Nous nous intéressons à la **reconnaissance automatique des panneaux de signalisation** pour la conduite autonome, à partir du jeu de données GTSRB (German Traffic Sign Recognition Benchmark) qui contient 43 classes de panneaux routiers.

Ce choix est motivé par la place centrale de la perception visuelle dans les véhicules autonomes, un système de conduite doit être capable de détecter et de classer rapidement les panneaux, malgré les variations de luminosité, d'angle de vue ou de qualité d'image. Ce projet nous permet également de mettre en pratique les notions de calcul haute performance vues en cours (utilisation du GPU, mini-batches, data augmentation à la volée).

1.1 Objectifs du projet

Les objectifs principaux sont :

- Mettre en place un pipeline complet de deep learning : prétraitement, entraînement, évaluation et inférence.
- Concevoir une architecture de CNN adaptée au problème multi-classes GTSRB.
- Analyser finement les performances du modèle et les confusions entre panneaux visuellement proches.

2 Jeu de données GTSRB

2.1 Description générale

Le benchmark GTSRB contient des images couleur de panneaux de signalisation allemands, réparties en 43 classes (limitations de vitesse, interdictions, obligations, panneaux de danger, etc). Les images présentent des variations importantes de taille, de luminosité, d'angle de vue et parfois de flou, ce qui en fait un cas d'étude réaliste pour la conduite autonome.

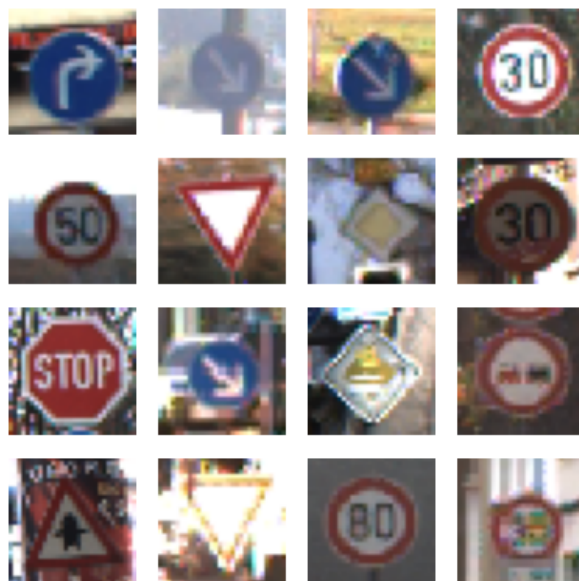


FIGURE 1 – Exemples d'images du jeu de données GTSRB. Les panneaux sont de petite taille (30×30) et présentent des variations de luminosité et de netteté, ce qui rend la tâche de classification plus difficile pour le modèle.

Le jeu de données officiel comporte 39 209 images d'entraînement et 12 630 images de test. Dans ce projet, nous effectuons un découpage interne de l'ensemble d'entraînement en 31 368 images pour l'entraînement et 7 842 images pour la validation (soit 20 % de validation), tout en conservant les 12 630 images du jeu de test pour l'évaluation finale.

2.2 Prétraitement des données

Dans ce projet, chaque image est redimensionnée en 30×30 pixels avec trois canaux couleur (RGB). Les valeurs de pixels sont normalisées dans l'intervalle $[0, 1]$ et une séparation est effectuée entre ensembles d'entraînement, de validation et de test.

Nous appliquons une légère augmentation de données (rotations, zooms modérés, translations horizontales/verticales, etc.) afin d'améliorer la robustesse du modèle aux variations de prise de vue.

Ce choix d'augmentation reste volontairement raisonnable pour ne pas déformer les panneaux de manière irréaliste (pas de *flip* horizontal, pas de rotations trop fortes).

3 Méthodologie et architecture du modèle

3.1 Pipeline général

Le pipeline d'apprentissage profond mis en place comprend les étapes suivantes :

1. Chargement et prétraitement des images GTSRB.
2. Définition de l'architecture CNN sous Keras / TensorFlow.
3. Entraînement du modèle sur l'ensemble d'entraînement, avec suivi des performances sur un ensemble de validation.
4. Évaluation finale sur le jeu de test et analyse des erreurs (matrice de confusion, rapport de classification).
5. Visualisation de prédictions correctes et incorrectes pour une analyse qualitative.

3.2 Architecture du réseau de neurones convolutionnel

Le modèle que nous avons utilisé est un CNN construit *from scratch*, adapté aux images de petite taille $30 \times 30 \times 3$.

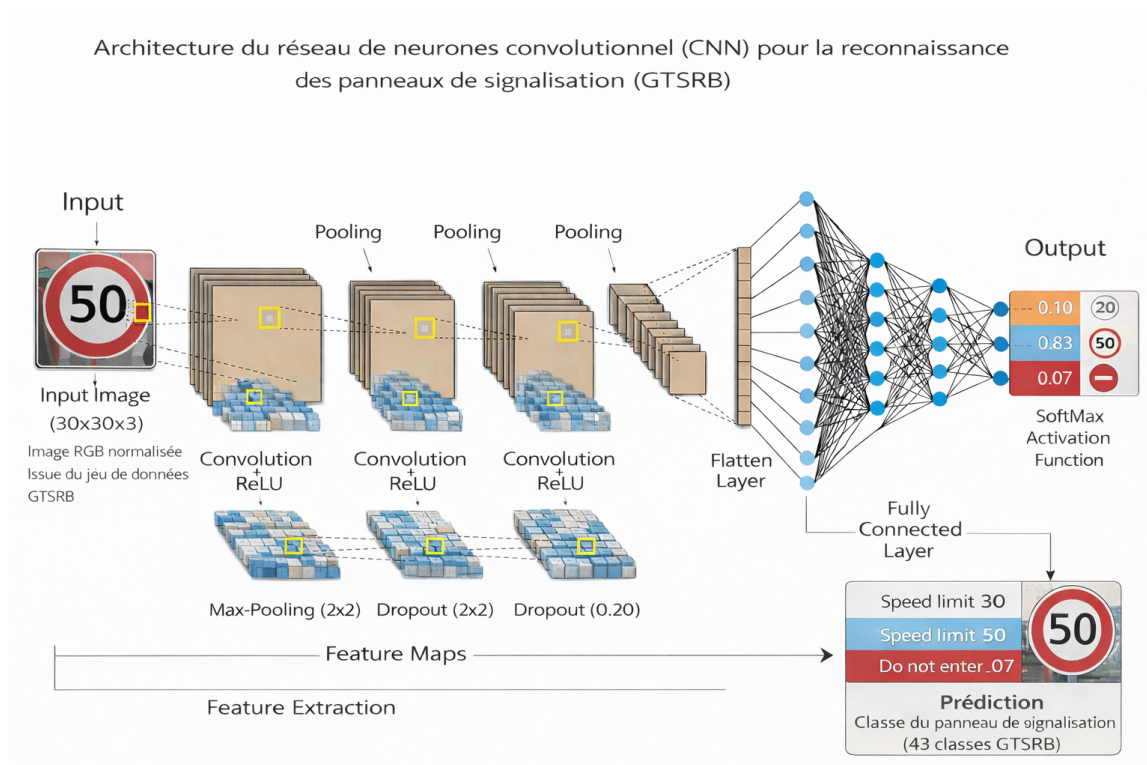


FIGURE 2 – Schéma conceptuel du CNN (entrée $30 \times 30 \times 3$, sortie : 43 classes GTSRB).

Après avoir représenté le pipeline global figure 2 nous passons à l'architecture concrète que nous avons réellement implémentée sous TensorFlow/Keras. L'idée est simple, les couches convolutionnelles extraient progressivement des caractéristiques (bords, formes, motifs), puis la couche **Flatten** transforme ces cartes en un vecteur exploité par les couches denses pour produire une probabilité par classe via **softmax**. Cette vue conceptuelle nous aide à relier chaque bloc à son rôle, avant de détailler précisément les dimensions et le nombre de paramètres.

La figure 3 présente ensuite le résumé de l'architecture généré par Keras (`model.summary()`).

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 30, 30, 3)	0
data_augmentation (Sequential)	(None, 30, 30, 3)	0
conv2d (Conv2D)	(None, 30, 30, 32)	2,432
conv2d_1 (Conv2D)	(None, 30, 30, 64)	51,264
max_pooling2d (MaxPooling2D)	(None, 15, 15, 64)	0
dropout (Dropout)	(None, 15, 15, 64)	0
conv2d_2 (Conv2D)	(None, 15, 15, 128)	73,856
conv2d_3 (Conv2D)	(None, 15, 15, 256)	295,168
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 256)	0
dropout_1 (Dropout)	(None, 7, 7, 256)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 512)	6,423,040
dropout_2 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 43)	22,059

FIGURE 3 – Résumé de l'architecture du modèle CNN (sortie de `model.summary()`). On observe deux blocs convolutionnels suivis d'une couche dense de 512 neurones, ce qui permet de construire progressivement des caractéristiques visuelles de plus en plus abstraites.

À partir de ce résumé nous vérifions deux points essentiels : la compatibilité des dimensions entre blocs (convolutions/pooling puis vectorisation), et la taille du modèle en nombre de paramètres, car elle impacte directement le temps d'entraînement et l'utilisation mémoire sur GPU. Nous constatons que l'essentiel des paramètres se situe dans la partie dense, ce qui est classique lorsque l'on utilise une couche `fully connected` relativement large après `Flatten`.

L'architecture se compose des éléments principaux suivants :

- Une couche de *data augmentation* appliquée aux images d'entrée.
- Deux blocs convolutionnels :
 - Bloc 1 : deux couches `Conv2D` (32 et 64 filtres) suivies d'une couche `MaxPooling2D` et d'un `Dropout`.
 - Bloc 2 : deux couches `Conv2D` (128 et 256 filtres) suivies d'une couche `MaxPooling2D` et d'un `Dropout`.
- Une couche `Flatten` pour vectoriser les cartes de caractéristiques.
- Une couche dense de 512 neurones avec activation `ReLU` et `Dropout`.
- Une couche de sortie dense de 43 neurones avec activation `softmax` pour la classification multi-classes.

Le modèle contient au total **6 867 819 paramètres** (environ 26,2 MB), tous entraînaibles, ce qui permet d'apprendre des représentations visuelles riches tout en restant compatible avec une exécution sur GPU Colab. Nous avons volontairement limité la profondeur du réseau pour garder un temps d'entraînement raisonnable dans le cadre du module.

3.3 Justification des choix techniques

Nous avons choisi un modèle *from scratch* plutôt qu'un réseau pré-entraîné (transfer learning) afin de contrôler précisément la taille du modèle et de mieux comprendre le rôle de chaque couche dans la reconnaissance des panneaux. Les couches `Conv2D` avec de petits filtres 3×3 apprennent des motifs locaux (bords, contours, formes simples), tandis que les blocs successifs et le `MaxPooling2D` construisent progressivement des caractéristiques plus globales.

L'utilisation de `Dropout` entre les blocs convolutionnels et avant la couche dense de 512 neurones vise à limiter le surapprentissage, en particulier parce que certaines classes de GTSRB sont peu représentées. Enfin, nous avons retenu l'optimiseur Adam pour sa capacité à s'adapter automatiquement au gradient de chaque paramètre et sa bonne performance empirique sur les CNN pour la vision par ordinateur.

Nous nous sommes limités à 10 époques d'entraînement afin de rester dans des temps de calcul compatibles avec une séance de travail sur GPU Colab ; des essais préliminaires ont montré que les courbes de loss et d'accuracy se stabilisaient déjà autour de cette valeur.

3.4 Hyper-paramètres d'entraînement

L'entraînement est effectué avec les choix suivants :

- Optimiseur : `Adam`.
- Fonction de perte : entropie croisée catégorielle.
- Taille de batch : 64.
- Nombre d'époques : 10.
- Callbacks : `EarlyStopping` (patience sur la loss de validation) et `ModelCheckpoint` pour sauvegarder le meilleur modèle durant l'entraînement.

Ces valeurs offrent un bon compromis entre stabilité de l'entraînement et temps de calcul sur GPU.

4 Résultats expérimentaux

4.1 Évolution de l'accuracy et de la loss

La figure 4 présente l'évolution de l'accuracy et de la fonction de perte sur les ensembles d'entraînement et de validation au cours des époques.

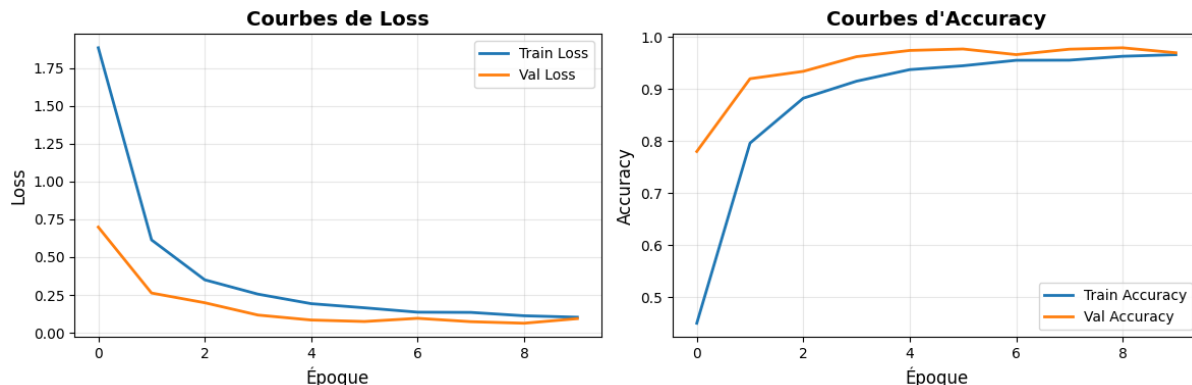


FIGURE 4 – Loss (gauche) et accuracy (droite) sur les ensembles d'entraînement et de validation. Les deux courbes restent proches, ce qui indique une bonne généralisation du modèle sans surapprentissage marqué.

L'accuracy d'entraînement progresse rapidement jusqu'à environ 0,98, tandis que l'accuracy de validation atteint environ **0,97** et reste proche de la courbe d'entraînement.

De même, la loss d'entraînement et la loss de validation diminuent toutes deux avant de se stabiliser à des valeurs faibles, sans divergence visible entre les deux courbes.

Ces observations indiquent une **bonne convergence** du modèle et un surapprentissage limité : le réseau apprend à généraliser sans simplement mémoriser l'ensemble d'entraînement.

4.2 Performances globales sur le test

Sur le jeu de test, le modèle atteint une **accuracy de 0,9050**, avec une macro-F1 d'environ **0,85** et une F1 pondérée proche de **0,90**. La macro-F1 mesure la performance moyenne sur l'ensemble des classes, tandis que la F1 pondérée tient compte du déséquilibre du dataset ; le fait que les deux valeurs soient proches montre que le modèle ne se contente pas de bien prédire uniquement les classes majoritaires.

Ces résultats sont légèrement inférieurs à ceux de validation, ce qui est attendu sur un vrai jeu de test plus difficile. ils restent néanmoins cohérents avec la littérature pour des architectures CNN de complexité modérée appliquées à GTSRB.

4.3 Matrice de confusion et analyse des confusions

La matrice de confusion sur le jeu de test est présentée figure 5. La majorité de la masse se trouve sur la diagonale, ce qui indique que la plupart des panneaux sont correctement reconnus.

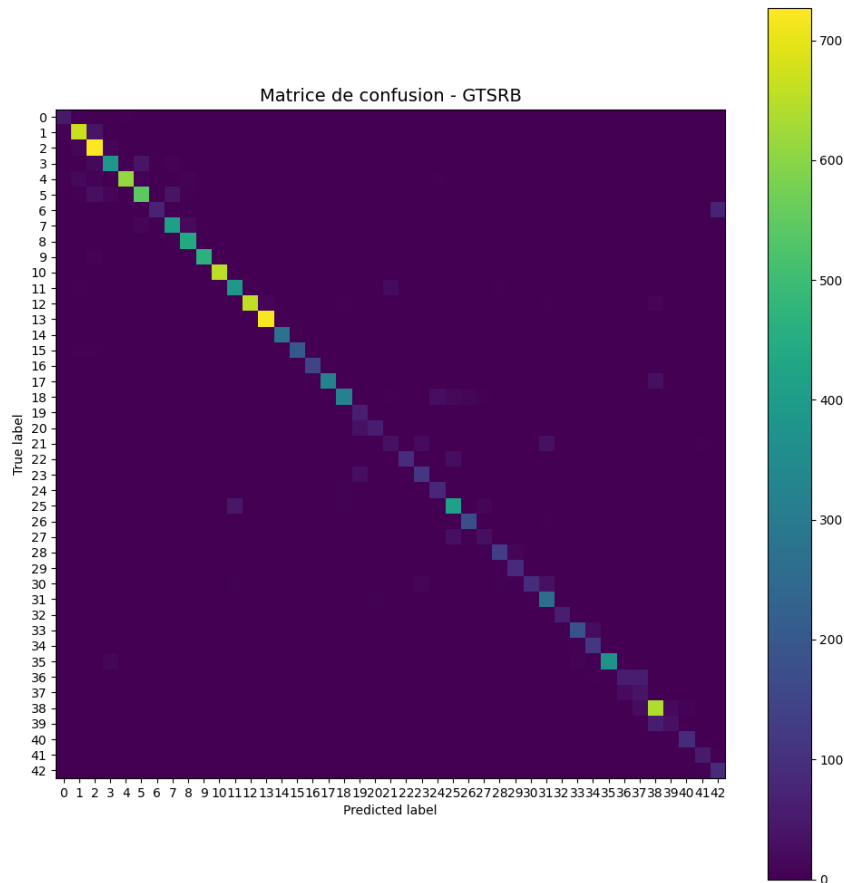


FIGURE 5 – Matrice de confusion du modèle sur le jeu de test GTSRB (43 classes). Les confusions se concentrent principalement entre panneaux de formes similaires (limitations de vitesse, panneaux triangulaires de danger).

On observe toutefois des confusions entre certains panneaux visuellement proches, par exemple entre plusieurs limitations de vitesse (panneaux circulaires rouges avec un nombre) ou entre certains panneaux de danger triangulaires. Ces erreurs sont cohérentes avec la difficulté du problème : lorsque le panneau est petit, flou ou partiellement occulté, même un humain peut hésiter entre deux classes voisines.

Les cases correspondant à des classes rares montrent également un nombre d'erreurs relativement plus élevé, ce qui confirme l'impact du déséquilibre du dataset. Cela justifie les pistes d'amélioration proposées plus loin, comme l'augmentation ciblée des classes minoritaires ou l'utilisation de poids de classes dans la fonction de perte.

4.4 Exemples qualitatifs de prédictions

Pour illustrer qualitativement le comportement du modèle, les figures 6 et 7 présentent respectivement des panneaux bien classés et mal classés.

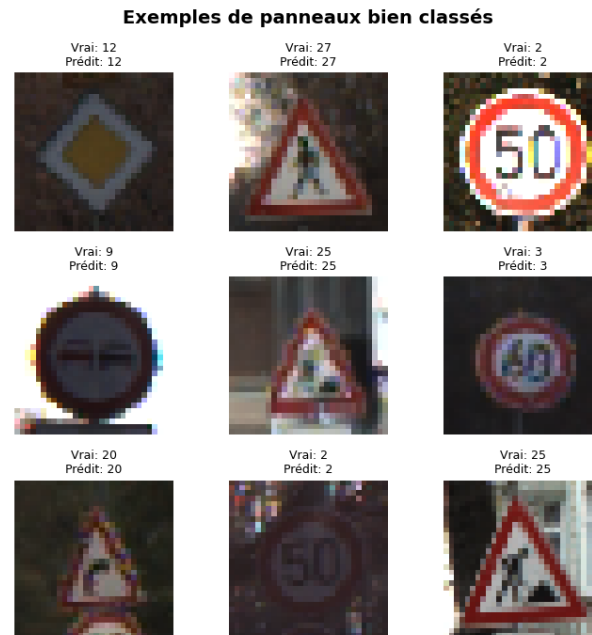


FIGURE 6 – Exemples de panneaux bien classés (Vrai = Prédit). On voit que le modèle gère correctement des conditions difficiles (panneaux sombres, léger flou ou contraste faible).

Dans cette figure, le modèle prédit correctement la classe de panneaux parfois peu contrastés, flous ou partiellement occultés, ce qui montre sa capacité à extraire des caractéristiques robustes malgré la faible résolution 30×30 .



FIGURE 7 – Exemples de panneaux mal classés (Vrai $\neq$ Prédit). Ces cas illustrent les limites actuelles du modèle sur des images très bruitées, sombres ou ambiguës.

À l'inverse, les exemples mal classés mettent en évidence plusieurs situations difficiles, Panneaux très sombres, fort flou de mouvement, ou classes visuellement très proches (par exemple différents panneaux de limitation de vitesse ou de danger). Ces erreurs suggèrent qu'une augmentation de données plus riche (variations de luminosité, bruit, contraste) ou des architectures plus profondes pourraient encore améliorer les performances sur ces cas extrêmes.

4.5 D  mo d'inf  rence sur une image du jeu de test

en compl  ment des m  triques globales et des grilles d'exemples (figures 6 et 7), nous avons r  alis   une d  monstration d'inf  rence sur une image du jeu de test s  lectionn  e al  atoirement.

Le r  seau produit une distribution de probabilit  s via **softmax** sur les 43 classes, et la classe pr  dite correspond    l'argmax. La Figure 8 affiche l'image, le label r  el, le label pr  dit et la confiance associ  e (probabilit   maximale).

Vrai: 38 | Pr  dit: 38 | Confiance: 0.993



FIGURE 8 – D  mo d'inf  rence sur une image du jeu de test : label r  el, label pr  dit et confiance (softmax).

5 Analyse critique et pistes d'am  lioration

Les r  sultats obtenus nous semblent satisfaisants pour une premi  re architecture construite *from scratch*, avec une accuracy de test d'environ 90 %. L'  cart observ   entre l'accuracy de validation (environ 0,97) et celle du jeu de test (environ 0,91) sugg  re que ce dernier contient des situations plus complexes, telles que des panneaux partiellement occult  s, flous ou de tr  s petite taille.

L'analyse de la matrice de confusion met en   vidence que les principales erreurs de classification concernent des classes visuellement proches, ainsi que certaines classes minoritaires. Ce comportement est typique des probl  mes de classification multi-classes avec un jeu de donn  es d  s  quilibr  .

Plusieurs pistes d'am  lioration peuvent   tre envisag  es :

- Augmenter la dur  e d'entra  nement (plus d'  poques) avec un ajustement dynamique du taux d'apprentissage (scheduler ou **ReduceLROnPlateau**).
- Enrichir l'augmentation de donn  es (variations de luminosit  , contrastes, l  g  res d  formations g  om  triques) pour mieux g  rer les conditions r  elles.
- Ajouter des couches de *Batch Normalization* apr  s les convolutions pour stabiliser l'entra  nement et am  liorer la g  n  ralisation.
- Traiter explicitement le d  s  quilibre de classes (par exemple via des poids de classes dans la loss ou une augmentation cibl  e des classes rares).
- Tester des architectures plus profondes ou du transfer learning    partir de mod  les pr  -entra  n  s sur ImageNet (par exemple ResNet ou MobileNet) afin de comparer les performances et le co  t de calcul.

6 Implémentation et aspects HPC

L'ensemble du pipeline est implémenté en Python avec TensorFlow / Keras et exécuté sur un GPU via Google Colab. L'utilisation du GPU nous permet d'entraîner le modèle (6,8 millions de paramètres) en un temps raisonnable sur 10 époques, ce qui serait beaucoup plus long sur CPU uniquement.

Les mini-batches de taille 64 exploitent le parallélisme massif du GPU pour traiter plusieurs images en une seule passe, tandis que la *data augmentation* est effectuée à la volée pour limiter l'empreinte mémoire. Même si l'objectif principal du projet est pédagogique, cette implémentation illustre bien le lien fort entre calcul haute performance et entraînement de modèles de deep learning sur des jeux de données d'images réels.

7 Conclusion

Ce projet nous a permis de mettre en place un pipeline complet de classification de panneaux de signalisation avec un réseau de neurones convolutionnel, depuis le prétraitement des images GTSRB jusqu'à l'inférence sur de nouvelles images pour la démonstration. Le modèle développé atteint de bonnes performances (accuracy test de 0,9050, macro-F1 de 0,85 et F1 pondérée de 0,90) et met en évidence les enjeux liés aux variations d'illumination, d'angle et au déséquilibre des classes.

Au-delà des résultats numériques, ce travail nous a permis de comprendre concrètement le rôle de chaque bloc du CNN (convolutions, pooling, dropout, couche dense) et l'impact des choix d'hyper-paramètres sur la convergence et la généralisation. Les pistes d'amélioration proposées (augmentation de données plus riche, Batch Normalization, gestion du déséquilibre, éventuel transfer learning) ouvrent la voie à des travaux futurs à rapprocher le modèle des performances de l'état de l'art, tout en respectant les contraintes du module HPC pour l'Intelligence Artificielle.